ZTaint-Havoc: From Havoc Mode to Zero-Execution Fuzzing-Driven Taint Inference

YUCHONG XIE
Hong Kong University of Science and Technology China

WENHUI ZHANG
Hunan University, China

DONGDONG SHE
Hong Kong University of Science and Technology, China

ISSTA 2025

백가현
2026711980

Motivation – Hot Bytes

- Core challenge in Fuzzing: **Hot Bytes**
  - Hot Byte - 분기 결정에 영향을 미치는 값

- Previous work
  - Dynamic Taint Analysis (**DTA**)
    - 정확
    - 오버헤드가 크고 대규모 프로그램에 부담
  - Fuzzing-Driven Taint Inference (**FTI**)
    - DTA 보다 가볍지만 보통 추가 실행이 많이 필요하다.

- **FTI**
  - 원본 입력 x, 변이 입력 x′ 실행.
  - Branch 의 conditional variable value 또는 branch behavior 를 비교
  - 문제
    - 바이트 단위로 따로따로 실험 -> 실행 비용이 커짐
- 효과적인 FTI 의 2 가지 요구사항
  1. Minimal Difference between program and mutant input
  2. Thorough program input byte examination

Motivation – Hot Bytes

- **Havoc Mode**
  - seed 에 여러 unit mutation 을 적용해 새로운 testcase 를 생성하는 AFL 계열의 핵심 Mutation 단계
  - Bit flip, byte flip, 증가/감소, overwrite, splicing 등을 사용

- Havoc Mode 는
  1. Single-byte 수준의 작은 변경도 만들 수 있고
  2. 입력 전역에서 mutation 위치를 고르게 선택하는 성질이 있어
  - **FTI 의 두 요구 사항에 부합**

Background

- **Undertaint**

  Dynamic Taint Analysis (DTA) 에서
  정보 흐름을 제대로 추적하지 못하는 오류.

- **Overtaint**

  분석 도구가 오염된 것으로 잘못 판단하는 경우

- **Splicing**

  A common strategy in numerous fuzzers.

  -> **splicing stage, splicing operator**

$$\text{Havoc Mode} : m = \begin{cases} f^h(s) = (\underbrace{f(f(\cdots f(s)\cdots))}_{h \text{ times}}) & \text{Normal Stage} \\ f^h(s') = (\underbrace{f(f(\cdots f(s'))\cdots))}_{h \text{ times}}), s' = splice(s) & \text{Splicing Stage} \end{cases}$$

- **Havoc Stack**

  하나의 mutant 를 만들 때, 몇개의 unit mutation 을 연속으로 쌓을지 나타내는 수

  The total number of unit mutations to be applied incrementally to an initial seed when generating a new testcase.

- **Havoc Operator**

  입력 하나를 받아 작은 랜덤 변형 하나를 적용하는 기본 mutation 단위

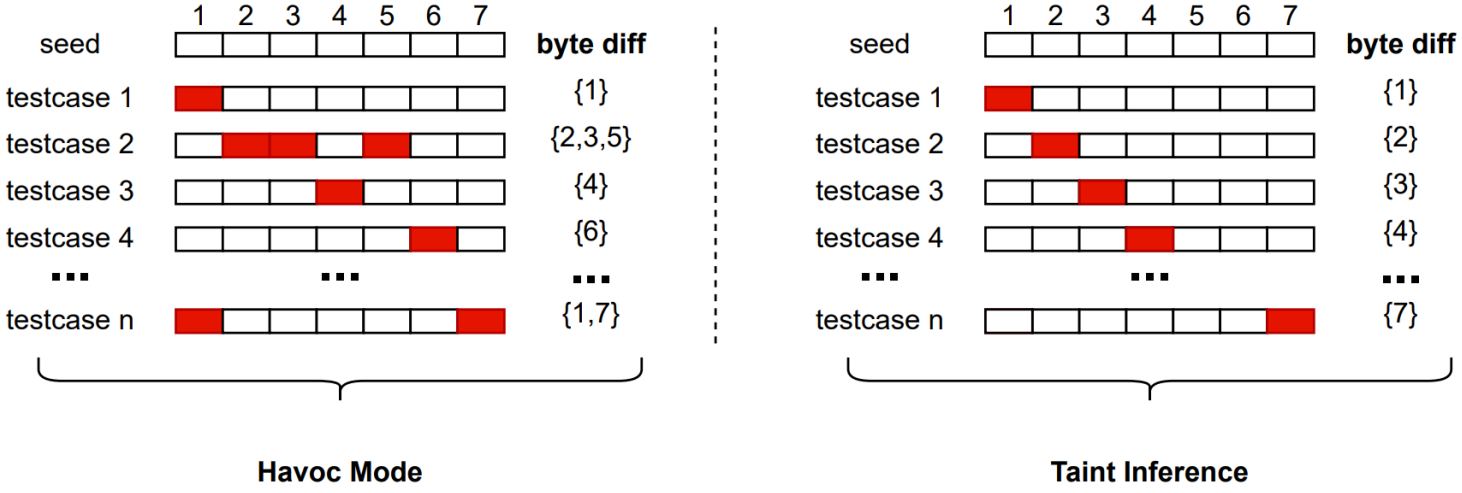
  A basic input mutation module in havoc mode.

```
u8  bit = rand_below(afl, 8);
u32 off = rand_below(afl, temp_len);
out_buf[off] ^= 1 << bit;
```

[a classic Havoc operator]

Connecting Havoc Mode with Fuzzing-Driven Taint Inference

- Havoc mode 가 대량의 mutant 를 만들다 보면 거의 모든 바이트 위치를 커버하게 된다.
- 특히 stack 이 작을 때는 single-byte difference 를 가진 mutant 도 충분히 생성된다.
- 따라서 havoc Mutation 하나하나를 FTI 의 inference instance 로 해석할 수 있다.
  - 효과적인 FTI를 위한 2가지 조건을 만족하며 **Zero-execution** FTI 가 가능하다.



Adaptive Threshold k

- **Mutation Threshold**
  - Seed 와 mutant 사이의 hamming distance 가 K 보다 크다면 inference 에서 제외시킨다.
  - $K = \text{seed}$ 길이와 mutation 수를 반영.

  - Adaptive threshold mechanism offers advantages
    - Length-Aware Sampling
    - Mutation-Aware Adjustment
    - Path Preservation
    - Distribution Normalization

$$D_{\text{Hamming}}(seed, mutant) \leq k$$

$$k = \beta \cdot \frac{L}{N_{mut}}$$

Cumulative Taint Inference

- 매 inference 결과를 바로 “정답 taint”로 쓰지 않음
- 대신 입력 길이만큼의 taint count array를 두고, 유효한 inference가 나올 때마다 해당 바이트의 count를 증가
- 여러 번 누적하면 진짜 hot byte는 계속 높은 count를 갖고, 우연한 noise는 상대적으로 약해짐.
  - The advantage of this approach lies in its ability to differentiate between true taints and noise.

=> Improving accuracy and mitigating overtaint issues.

algorithm

- **Phase 1: Sampling / Zero-execution FTI**
  - vanilla havoc로 mutant 생성
  - threshold 이내의 mutant만 보고
  - branch-related behavior가 달라지면 diff byte의 taint count 증가
- **Phase 2: Taint-guided Mutation**
  - count가 높은 바이트를 mutation 시작점으로 더 자주 선택
  - mutation을 **hot byte 중심으로 bias**함

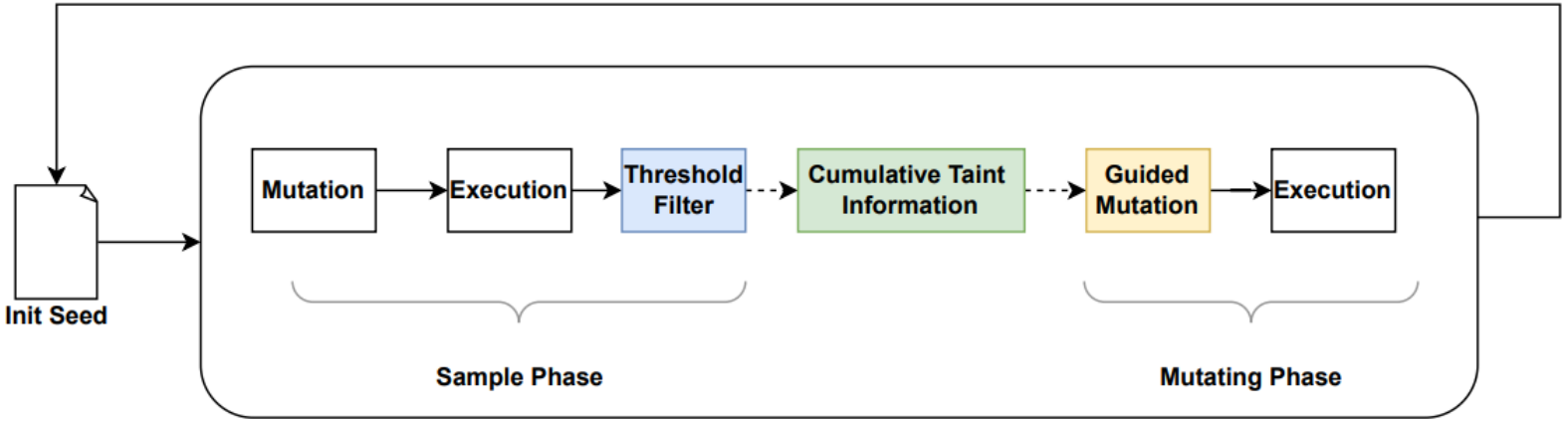


Fig. 1. **Workflow of ZTaint-Havoc.**

Implementation

- **Control Flow Graph**: 프로그램의 실행 그래프
- **Frontier node**: 일반 CFG 전체에서도, 이미 도달한 실행 영역과 아직 도달하지 못한 실행 영역의 경계에 놓인 노드
  - 현재까지 한 번 이상 방문한 노드임에도 자식 중 아직 방문하지 않은 노드가 적어도 하나 있는 노드.
- **Frontier branch**: Ztaint-Havoc 에서는 frontier node 전체를 사용하지 않고, 그 중에서도 조건 분기와 직접 연결된 것만 더 좁혀 쓴다.
  - Frontier node 의 부분 집합.
- strcmp, memcmp 와 icmp 를 다르게 다루는 이유
  - icmp: LLVM IR 의 단일 비교 instruction => inserting single instruction
  - strcmp, memcmp: 함수 호출 => inserting a function

```
if (buf[3] == 'A') {
    // 아직 안 들어와 본 경로
} else {
    // 지금까지는 항상 여기만 실행됨
}
```

Implementation

- AFL++ 4.10c 기반으로 구현
- LLVM instrumentation을 확장
  - intraprocedural CFG (함수 하나 내부만 대상으로 만든 CFG)와 frontier branch를 관리
- Icmp, strcmp, memcmp 등의 비교 지점에서 **control condition 변화**를 추적
- 오버헤드를 줄이기 위해 hook은 frontier branch에만 adaptive하게 활성화
- havoc 단계에서 앞 50% mutation을 sampling에 사용하고, 이후 taint-guided mutation으로 전환

Algorithm 1 Enhanced Havoc Mode with Zero-Execution FTI

```
Initialize taint_count[1..n] ← 0 where n is input length
seed → initial seed
/* Phase 1: Vanilla havoc mode with zero-execution FTI */
pb_seed = execute(seed)                          ▷ Record program behavior for seed
for i = 1, 2, ..., n_1 do
    t_i = havoc_mutate(seed)                     ▷ Vanilla havoc mode generates a testcase t_i
    pb_i = execute(t_i)                          ▷ Record program behavior for t_i
    ByteDiff = diff(t_i, seed)
    if |ByteDiff| ≤ k then                       ▷ Skip testcases with too many byte diffs to avoid overtaint
        if pb_i ≠ pb_seed then
            for each index j in ByteDiff do
                taint_count[j] ← taint_count[j] + 1
/* Phase 2: Biased havoc mode focusing on bytes with high taint count */
for i = 1, 2, ..., n_2 do
    t_i = biased_havoc_mutate(seed, taint_count) ▷ Mutate bytes based on taint count
    execute(t_i)
```

Experimental setting

- 벤치마크: FuzzBench 19개 + UniBench 17
- **Research Question**:
  - havoc가 정말 균등 분포 성질을 가지는가
  - threshold가 분포를 보정하는가
  - coverage는 AFL++보다 좋아지는가
  - overhead는 어느 정도인가

Table 1. **Studied programs in our evaluation.**

Targets	Version	# Edge	Targets	Version	# Edge	Targets	Version	# Edge
FuzzBench Targets			sql	c78cbf2	77,147	tcpdump	4.8.1	32,760
bloaty	52948c1	163,038	systemd	07faa49	1,059	jq	jq-1.5	6,177
zlib	zlib-1.2.13	4,634	jsoncpp	8190e06	9,820	sqlite3	SQLite-3.8.9	67,533
lcms	f0d9632	13,649	libjpeg	b0971e4	17,783	cflow	cflow-1.6	7,690
libpcap	c639612	15,450	libpng	cd0ea2a	9,032	exiv2	exiv2-0.28.0	122,536
libxml2	c7260a4	82,498	libxslt	180cdb8	60,091	ffmpeg	ffmpeg-4.0.1	586,737
openh264	fa6d099	18,677	openssl	b0593c0	85,304	infotocap	ncurses-6.1	11,470
re2	4be2407	15,173	**UniBench Targets**			nm-new	binutils-2.28	57,709
stbi	5736b15	7,655				objdump	binutils-2.28	77,318
vorbis	84c0236	12,895	mp42aac	1.5.1-628	21,707	pdftotext	xpdf-4.00	49,169
woff2	9476664	19,609	lame	lame-3.99.5	15,776	mp3gain	1.5.2	3,344
curl	a20f74a	122,327	mujs	mujs-1.0.2	21,288	tiffsplit	tiff-3.9.7	12,527
harfbuzz	cb47dca	78,196	flvmeta	flvmeta-1.2.1	6,512	jhead	3.00	2,092

RQ 1

- Does havoc mode follow a uniform distribution?

No constraints VS applying k=5

mutation 시작 위치는 전반적으로 균등 분포에 가까움

하지만 threshold 없이 보면 실제 influenced byte 분포는 비균등

threshold를 적용하면 influenced byte 분포가 훨씬 더 균등해짐

또한 havoc stack이 커질수록 perturbation distance도 커짐

Binary	Length	Binary	Length
cflow*	38,139	vorbis	2,603
bloaty	22,120	openssl	1,114
pdftotext*	12,567	tcpdump*	531
tiffsplit*	9,834	libpcap	220
ffmpeg*	6,714	lcms	128

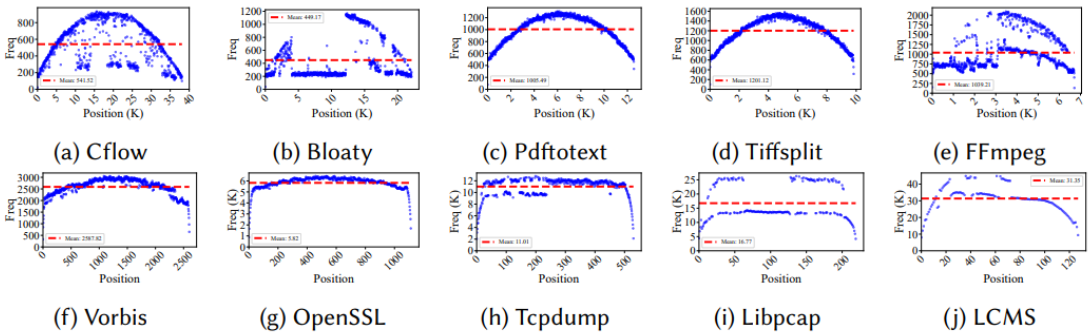


Fig. 4. **Distribution of influenced bytes (bytes that differ between the original seed and mutated input) across ten target programs. The x-axis represents the byte position in input space, while the y-axis shows the accumulated frequency of bytes being influenced over 10 independent runs, demonstrating the distribution pattern without threshold constraints. The red line indicates the average frequency across all positions.**

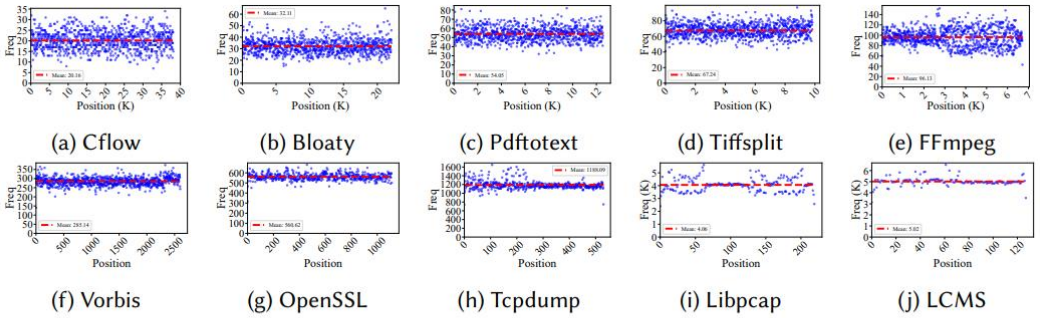


Fig. 5. **Distribution of influenced bytes across ten target programs with a perturbation threshold of k=5. The x-axis represents the byte position in input space, while the y-axis shows the accumulated frequency of bytes being influenced over 10 independent runs, demonstrating the distribution pattern under controlled mutation impact. The red line indicates the average frequency across all positions.**

RQ 2, 3

- Why is uniform distribution effective?

  Vanilla AFL++ VS AFL++

  -> compare the coverage

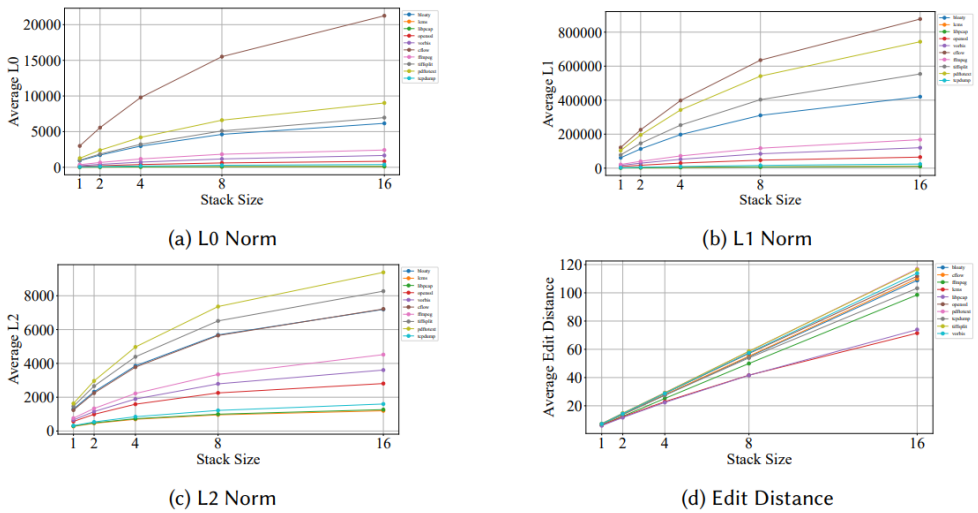
  vanilla AFL++의 **균등 sampling**은, 특정 중심 근처를 자주 고르는 정규분포형 sampling보다 평균 4.69% 더 높은 coverage를 보였다.

  -> 균등 분포가 입력 공간을 더 고르게 탐색하고, 프로그램 구조별 편향에 덜 취약하다.

- Effect of Havoc Stack Size

  havoc stack이 1, 2, 4, 8, 16으로 커질수록 L0 / L1 / L2 / edit distance가 모두 증가했다.

  -> **havoc stack**은 단순 반복 횟수가 아니라 **mutant가 원본에서 얼마나 멀어지는지를 결정**하는 핵심 인자다.



(a) L0 Norm (b) L1 Norm (c) L2 Norm (d) Edit Distance

RQ 4) Effect of Splicing

- splicing stage와 splicing operator는 일반 mutation보다 더 큰 perturbation distance를 만든다.
- 이는 더 과감한 탐색에는 유리하지만, zero-execution FTI에 그대로 쓰면 noise가 많아질 수 있다.

  => 따라서 **threshold** 같은 제어 장치가 중요하다.

Targets	L0-N	L0-S	L1-N	L1-S	L2-N	L2-S	Edit-N	Edit-S
cflow	10,238	15,041	418,382	673,027	3,802	6,168	36	8,898
bloaty	2,864	10,627	191,824	1,020,463	3,626	11,615	33	7,581
pdftotext	4,088	5,981	334,316	500,484	4,697	7,556	34	4,287
tiffsplit	3,745	5,606	295,852	476,705	4,852	7,468	44	3,013
ffmpeg	897	3,124	55,101	461,475	1,719	9,800	19	2,159
vorbis	736	1,587	53,111	139,519	1,841	4,081	34	1,194
openssl	402	741	30,973	60,058	1,562	2,714	35	489
tcpdump	149	304	8,345	23,111	699	1,594	26	232
libpcap	52	81	4,238	7,291	688	1,067	25	66
lcms	40	75	4,221	7,291	633	949	24	70

normal vs splicing stages

-N suffix: normal stage / -S suffix: splicing stage

Targets	L0-O	L0-S	L1-O	L1-S	L2-O	L2-S	Edit-O	Edit-S
cflow	2,308	7,565	93,400	306,402	971	3,047	6	15
bloaty	721	2,307	47,398	152,006	1,025	3,113	6	16
pdftotext	1,014	3,239	82,464	263,318	1,306	4,038	6	16
tiffsplit	780	2,472	61,489	195,378	1,169	3,586	6	17
ffmpeg	286	898	16,942	53,505	619	1,721	5	14
vorbis	172	555	12,380	39,841	524	1,555	6	16
openssl	93	301	7,124	23,232	453	1,335	6	16
tcpdump	44	140	2,318	7,300	264	685	6	16
libpcap	13	38	1,031	2,867	259	551	5	13
lcms	9	30	992	3,143	227	575	5	14

other operator vs splicing operator

-O suffix: mutation operator / -S suffix: splicing operator

RQ 5) Code Coverage Compared to AFL++

- Assess the coverage of AFL++ and ZTaint-Havoc under two different configuration
  - Default settings with no additional features
  - Enhanced settings with both dictionary and AFL++ cmplog mode enabled.
- 24시간, 10회 반복 실험에서FuzzBench 평균 +2.97% UniBench 평균 +6.12% edge coverage 향상을 보였다.
- **최대 향상은 FuzzBench: 33.71% (lcms) UniBench: 51.12% (nm-new)**

- Solver: 어떤 분기 조건을 통과하려면 입력을 어떻게 바꿔야하는지 계산해주는 구성 요소

RQ 6) Effect of Threshold Parameter k

- $\boldsymbol{k} = 1, 10, 100, 1000, 10000$ and **adaptive $\boldsymbol{k}$**
- 고정값 k보다 adaptive threshold가 전체 타깃에서 더 좋은 성능을 보였다.
- k가 너무 작으면 undertaint, 너무 크면 overtaint & overhead가 증가한다.
- 따라서 하나의 고정 threshold보다 adaptive 방식이 더 실용적이다.

Targets	1	10	100	1K	10K	Ada	Targets	1	10	100	1K	10K	Ada
curl	13,832	13,840	13,900	**14,059**	13,414	13,935	woff2	**2,409**	2,383	2,396	2,405	2,334	2,404
bloaty	3,088	**3,120**	3,050	3,086	3,103	3,093	jhead	357	357	358	**360**	357	**360**
harfbuzz	22,675	**22,820**	22,024	22,244	22,404	22,671	jq	3,378	3,348	3,351	3,352	3,335	**3,451**
zlib	**877**	**877**	**877**	**877**	**877**	**877**	sqlite3	12,835	12,995	11,849	**13,096**	12,642	12,574
jsoncpp	**1,337**	1,331	1,328	1,332	1,333	1,333	cflow	2,073	2,073	2,071	2,072	2,072	**2,097**
lcms	1,226	1,588	1,643	1,485	1,259	**1,683**	exiv2	2,103	2,233	2,116	2,106	**3,236**	2,662
libjpeg	3,208	3,213	3,154	3,071	**3,214**	3,198	ffmpeg	7,327	9,129	7,606	6,855	6,813	**13,933**
libpcap	47	47	42	44	47	**49**	infotocap	1,672	1,695	1,672	1,668	1,656	**1,718**
libpng	**1,405**	1,366	1,360	1,361	1,361	1,360	nm-new	1,505	1,567	1,514	1,538	1,543	**1,965**
libxml2	8,808	8,658	9,044	8,781	8,848	**9,546**	objdump	4,944	4,990	5,099	4,997	4,990	**5,215**
libxslt	4,681	4,732	**4,742**	4,645	4,639	4,659	pdftotext	6,239	6,252	6,256	**6,279**	6,229	6,267
openh264	13,640	13,608	13,148	**13,647**	13,628	13,591	mp3gain	1,429	1,415	1,412	1,406	**1,453**	1,424
openssl	**4,576**	4,554	4,550	4,569	4,550	4,559	tcpdump	2,916	3,029	3,283	2,998	3,082	**4,294**
re2	6,255	6,240	6,223	**6,269**	6,259	6,259	mujs	5,117	4,939	5,039	4,900	4,914	**5,456**
stbi	2,224	2,221	2,164	**2,332**	1,931	2,086	tiffsplit	1,096	1,355	1,293	1,201	1,206	**1,412**
sql	11,774	11,727	11,899	11,616	11,745	**11,932**	mp42aac	1,615	1,676	1,667	1,662	**1,682**	1,587
systemd	2,511	**2,522**	2,521	2,516	2,518	**2,522**	flvmeta	442	445	442	**446**	**446**	436
vorbis	2,042	**2,055**	2,049	2,049	2,043	2,037	**Glo_Win**	5	5	2	9	7	16
lame	5,639	5,654	5,653	5,611	**5,682**	5,679							

RQ 7) Runtime Overhead

- baseline 대비 overhead는 UniBench 3.84%, FuzzBench 12.58%
- FuzzBench에서 더 크게 보이는 이유는 원래 실행 속도가 빨라, fuzzer-side bookkeeping 비용이 상대적으로 더 크게 드러나기 때문이다.
- 그래도 저자들은 이를 실용적인 수준으로 본다.

Targets	Z-H	AFL++	Targets	Z-H	AFL++	Targets	Z-H	AFL++
FuzzBench Targets			sql	88,947	110,594	tcpdump	8,193	8,129
bloaty	31,982	29,848	systemd	24,597	27,272	jq	2,309	2,659
zlib	12,451	12,346	jsoncpp	104,963	135,009	sqlite3	1,896	1,901
lcms	215,976	306,414	libjpeg	651,025	708,923	cflow	3,666	3,686
libpcap	89,014	92,063	libpng	1,059,813	800,547	exiv2	4,769	5,828
libxml2	229,220	249,728	openssl	737,275	925,637	ffmpeg	877	1,124
openh264	3,998	3,997	**Overhead**	**3.84%**		infotocap	4,614	4,408
libxslt	497,771	530,097				jhead	12,860	12,642
re2	123,943	221,797	**UniBench Targets**			nm-new	6,446	7,183
stbi	35,938	44,769	objdump	5,947	6,110	tiffsplit	13,499	14,177
vorbis	273,842	304,640	mp42aac	9,063	9,835	pdftotext	4,328	4,417
woff2	305,926	444,382	lame	2,498	2,419	mp3gain	7,933	7,473
curl	514,058	711,715	mujs	8,685	8,894	**Overhead**	**12.58%**	
harfbuzz	332,316	411,216	flvmeta	16,235	15,883			

RQ 8) Comparison with CMPLOG

- branch solver를 붙인 ZTaint-Havoc with solver를 CMPLOG와 비교결과:
  - FuzzBench 평균 +3.06%, UniBench 평균 +13.54% coverage 향상
- 다만 일부 타깃에서는 CMPLOG가 더 우세했다.
- 저자들은 그 이유로 solver가 아직 simple prototype이라 float 처리나 변수-변수 string compare 같은 특수 처리가 부족하다고 설명

- CMPLOG: AFL++ 기능 중 하나로, 프로그램이 실행 중 수행하는 비교 정보를 기록해서 mutation을 더 똑똑하게 하도록 돕는 메커니즘

Discussion

- feedback, instrumentation을 요구한다.
- threshold value 선택이 여전히 중요하다.
- The use of filters to minimize noise in taint analysis leads to a smaller sample size for fuzzing-driven taint inference -> inference 효율이 떨어질 수 있다.

Conclusion

- Havoc mode 를 계산적으로 모델링하면 추가 실행 없는 FTI 가 가능하다.
- 실제 AFL++ 기반 구현에서도 coverage 향상이 확인됐다.
- 기본 fuzzing pipeline 안에 이미 들어 있는 havoc mutation 을 잘 해석하면, 비싼 taint inference를 거의 공짜처럼 얻을 수 있다.

Thank You